The transform_keys and transform_values methods are good alternatives to map if you only want to change part of the hash. Each takes a block, and transform_keys returns a new hash where each key/value pair has the same value as the original, but the key is the result of the block. The transform_values method is similar, but each resulting pair has the same key, and the value is the result of the block. Both methods have ! forms: transform_keys! and transform_values! that modify the original hash in place rather than return a new hash:

```
hash = {a: 1, b: 2, c: 3}
hash.transform_keys { |key| key.upcase }     # => {:A=>1, :B=>2, :C=>3}
hash.transform_values { |value| value ** 3 } # => {:a=>1, :b=>8, :c=>27}
```

Set

A Ruby Set is somewhere between an Array and a Hash—it's a collection of unordered, unique items. The Set class is a subclass of Object and it defines each and includes Enumerable, so all Enumerable methods described in this chapter apply to sets.

To use Set you need to require set—sets are part of Ruby but not part of the core Ruby that automatically gets loaded.

Creating Sets

There are several ways to create a Set. The Enumerable class defines to_set, which converts the collection into a set, removing duplicates along the way. The Set class has a unique API, where Set.new takes an Enumerable and converts it to a set. If you have just a bunch of objects and you want to make a set out of them, well the Set class implements [], so:

```
Set.new([1, 2, 3]) # => #<Set: {1, 2, 3}>
Set[1, 2, 3]       # => #<Set: {1, 2, 3}>
```

If you pass a value to Set.new that is not an Enumerable, you will get an ArgumentError. To be honest, it takes a little getting used to the second form.

Set elements are unique and the unique value is based on Object#hash, which under normal circumstances is the same as eql?. (It's the same logic that Hash uses for unique keys.) You can change that for an individual set with the compare_by_identity method, which changes to compare on internal object ID.

Modifying Sets

Because sets are unordered, modifying a set generally means adding to one or removing data from one. In addition to the methods provided by Enumerable, Set provides a few.

You can add a single object to a Set with the add method, which is also aliased as <<. If you want to add more than one item, you use merge, which takes one or more Enumerable objects as arguments and adds their elements one by one to the Set. The add? method takes one argument. If that argument is not in the set, the argument is added and the set is returned. If the argument is already in the set, nothing happens and the method returns nil. This is a deviation from the idiomatic Ruby practice of returning literal true or false from a question mark method.

To remove an object from a set, the delete method removes the object from the and returns the updated set. The similar method delete? removes the object and returns the set if the object is in the set, otherwise it does nothing and returns nil. If you want to remove every